

Bootcamp: Full Stack Junior

Modulo 1: Gestión Integral de Bases de Datos

Semana 1

Contenido

1

Modelado entidad-relación

2

Normalización de bases de datos

3

Fundamentos de bases de datos relacionales

4

SQL intermedio

Contenido:

1. Fundamentos de bases de datos

Una base de datos es un sistema organizado de información que permite almacenar, gestionar y recuperar datos de forma eficiente y estructurada. Funciona como un repositorio digital donde la información se organiza en tablas, registros y campos relacionados entre sí, facilitando su acceso, actualización y consulta.

Existen diferentes tipos de bases de datos: las relacionales (como MySQL o PostgreSQL) que organizan los datos en tablas conectadas mediante claves; las NoSQL (como MongoDB) diseñadas para datos no estructurados; y las orientadas a grafos para relaciones complejas, entre otras.

Los sistemas de gestión de bases de datos (DBMS) son el software que administra estas estructuras, garantizando la integridad, seguridad y consistencia de los datos. Permiten realizar operaciones mediante lenguajes como SQL para consultar, insertar, actualizar o eliminar información.

Las bases de datos son fundamentales en aplicaciones modernas: desde sistemas bancarios y plataformas de comercio electrónico hasta redes sociales y aplicaciones móviles. Su importancia radica en eliminar la redundancia de información, mantener la coherencia de los datos, permitir acceso concurrente de múltiples usuarios y garantizar respaldos para prevenir pérdidas.

En esencia, una base de datos es el corazón informático de cualquier sistema que necesite almacenar y procesar información de manera confiable y escalable.



Ilustración 1 - Tipos de IA - Fuente: Gemini Nano Banana Pro

2. Modelado Entidad-Relación (ER)

El Diagrama Entidad-Relación (DER o ER) es una herramienta de modelado visual que representa la estructura lógica de una base de datos. Fue creado por Peter Chen en 1976 y utiliza símbolos gráficos para mostrar cómo las diferentes entidades (objetos o conceptos) se relacionan entre sí dentro de un sistema de información.

1. Componentes principales de un diagrama Entidad-Relación (ER)

Concepto	Definición	Representación Gráfica	Ejemplos
Entidad	Objetos o conceptos del mundo real sobre los que se desea almacenar datos.	Rectángulo	Cliente, Producto, Empleado, Pedido.
Atributo	Características o propiedades que describen a una entidad.	Óvalo	Nombre, DNI, Precio, Fecha de nacimiento.
Relación	Vínculo o conexión lógica que une a dos o más entidades.	Rombo	"Compra", "Trabaja en", "Posee".

Cardinalidad	Indica el número de instancias de una entidad que pueden asociarse con otra.	Notación (1:1, 1:N, N:M)	Un cliente (1) hace muchos (N) pedidos.
--------------	--	--------------------------	---

Tabla 1 - Componentes de diagrama ER - Fuente: Autor de contenidos

2. Modelado Entidad-Relación (ER)

- **Comunicación efectiva:** Facilita la comunicación entre analistas, desarrolladores y clientes al proporcionar un lenguaje visual común que todos pueden entender, independientemente de sus conocimientos técnicos.
- **Planificación y diseño:** Permite planificar la estructura de la base de datos antes de su implementación, identificando problemas potenciales en etapas tempranas cuando son más fáciles y económicos de resolver.
- **Evita redundancia y errores:** Ayuda a identificar y eliminar datos duplicados, asegurando la integridad y consistencia de la información desde el diseño inicial.
- **Documentación:** Sirve como documentación permanente del sistema, facilitando el mantenimiento, actualizaciones futuras y la incorporación de nuevos desarrolladores al proyecto.
- **Base para la implementación:** Actúa como puente entre los requisitos del negocio y la implementación técnica, permitiendo convertir el modelo conceptual en tablas y relaciones físicas en el sistema gestor de bases de datos.
- **Optimización:** Permite visualizar y optimizar las relaciones entre datos, mejorando el rendimiento y la eficiencia de las consultas futuras.

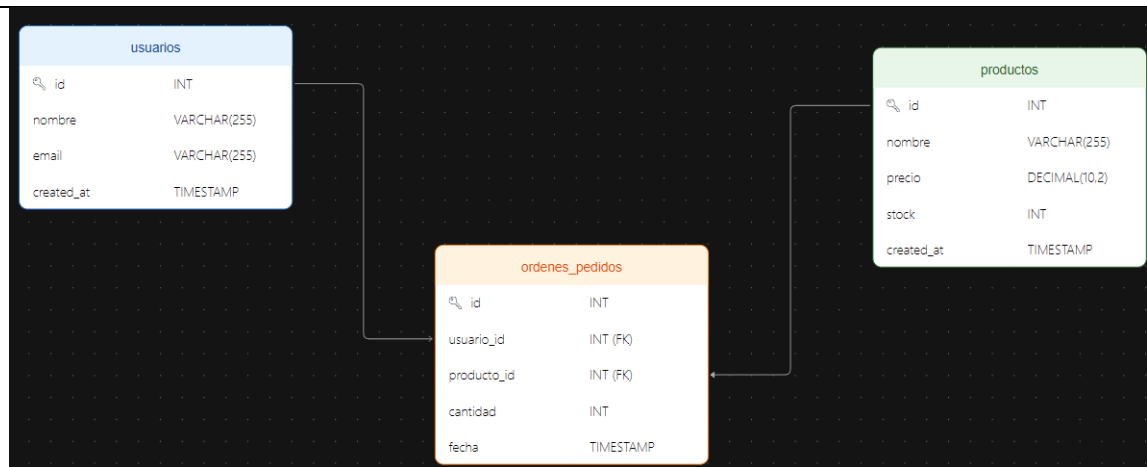


Ilustración 2 - Ejemplo diagrama ER - Fuente: Autor de contenidos

3. Normalización de base de datos

La normalización es un proceso sistemático de organizar los datos en una base de datos para reducir la redundancia, eliminar anomalías y garantizar la integridad de los datos. Consiste en aplicar una serie de reglas (formas normales) que estructuran las tablas de manera óptima.

1. Objetivos de la Normalización

- **Eliminar redundancia:** Evitar almacenar la misma información en múltiples lugares
- **Prevenir anomalías:** Evitar inconsistencias al insertar, actualizar o eliminar datos
- **Optimizar el espacio:** Reducir el almacenamiento innecesario
- **Facilitar el mantenimiento:** Hacer los cambios más simples y seguros
- **Garantizar integridad:** Asegurar que los datos sean precisos y confiables

2. Formas Normales

Primera Forma Normal (1FN)

Requisitos:

- Todos los atributos deben contener valores atómicos (indivisibles)
- No puede haber grupos repetitivos o conjuntos de valores múltiples
- Cada columna debe tener un único tipo de dato
- Debe existir una clave primaria que identifique únicamente cada registro

Ejemplo de violación (1FN):

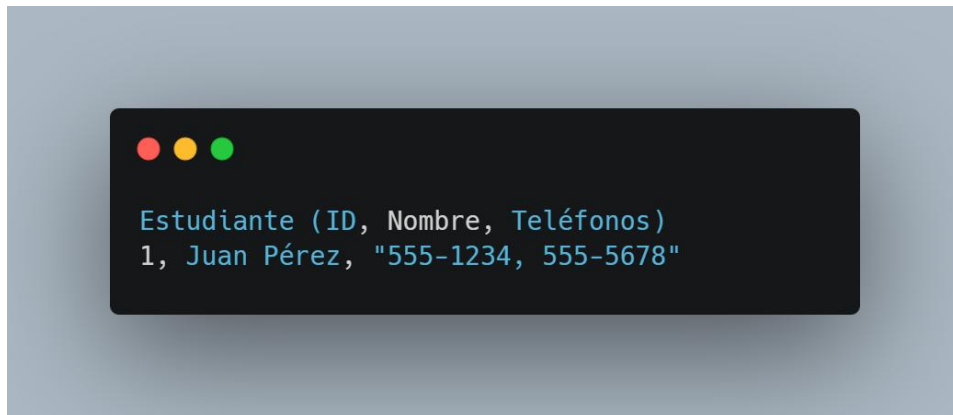


Ilustración 3 - Ejemplo violación 1FN - Fuente: Autor de contenido

Solución (1FN):

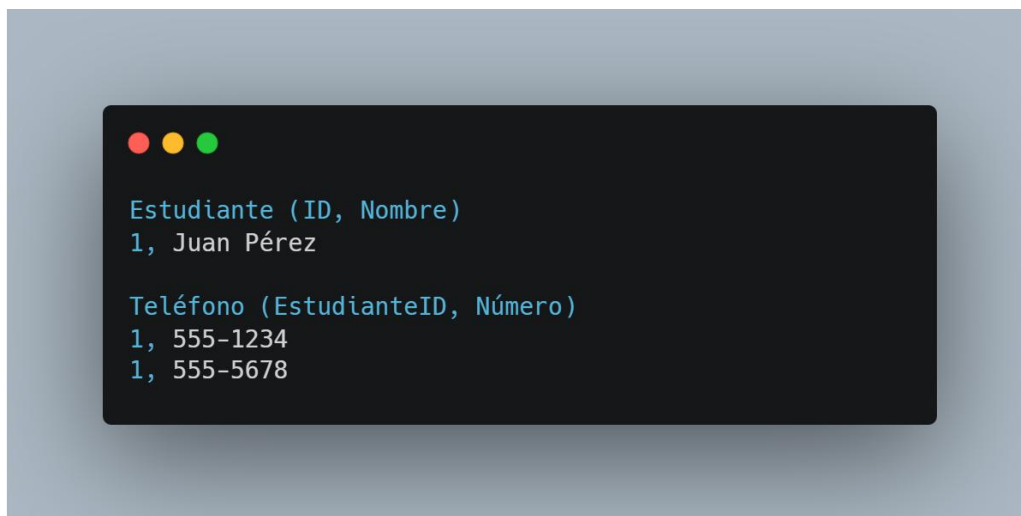


Ilustración 4 - Ejemplo solución 1FN - Fuente: Autor de contenido

Segunda Forma Normal (2FN)

Requisitos:

- Debe cumplir con 1FN
- Todos los atributos no clave deben depender completamente de la clave primaria completa (no de parte de ella)
- Aplica principalmente a tablas con claves primarias compuestas

Ejemplo de violación (2FN):



```
Inscripción (EstudianteID, CursoID, NombreEstudiante, NombreCurso, Calificación)
Clave primaria: (EstudianteID, CursoID)
```

Ilustración 5 - Ejemplo violación 2FN - Fuente: Autor de contenido

Problema: NombreEstudiante solo depende de EstudianteID, y NombreCurso solo depende de CursoID.

Solución (2FN):



```
Estudiante (EstudianteID, NombreEstudiante)
Curso (CursoID, NombreCurso)
Inscripción (EstudianteID, CursoID, Calificación)
```

Ilustración 6 - Ejemplo solución 2FN - Fuente: Autor de contenido

Tercera Forma Normal (3FN)

Requisitos:

- Debe cumplir con 2FN
- No debe haber dependencias transitivas (un atributo no clave no debe depender de otro atributo no clave)
- Todos los atributos no clave deben depender directamente de la clave primaria

Ejemplo de violación:



```
Empleado (EmpleadoID, Nombre, DepartamentoID, NombreDepartamento)
```

Ilustración 7 - Ejemplo violación 3FN - Fuente: Autor de contenido

Problema: NombreDepartamento depende de DepartamentoID, no directamente de EmpleadoID.

Solución (3FN):



```
Empleado (EmpleadoID, Nombre, DepartamentoID)
Departamento (DepartamentoID, NombreDepartamento)
```

Ilustración 8 - Ejemplo solución 3FN - Fuente: Autor de contenido

Forma Normal de Boyce-Codd (FNBC)

Requisitos:

- Debe cumplir con 3FN
- Para cada dependencia funcional $X \rightarrow Y$, X debe ser una superclave
- Es una versión más estricta de 3FN

Ejemplo de violación:

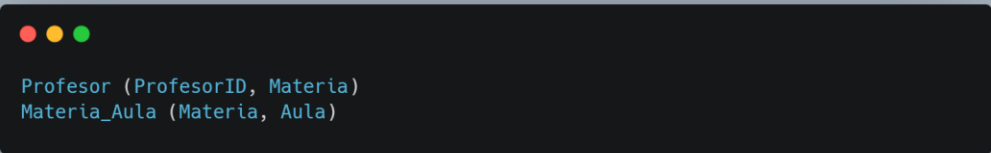


```
Profesor_Materia (ProfesorID, Materia, Aula)
```

Ilustración 9 - Ejemplo violación FNBC - Fuente: Autor de contenido

Si un profesor solo puede enseñar una materia, pero una materia puede tener varios profesores, y cada materia se enseña en un aula específica.

Solución (FNBC):



```
Profesor (ProfesorID, Materia)
Materia_Aula (Materia, Aula)
```

Ilustración 10 - Ejemplo solución FNBC - Fuente: Autor de contenido

Cuarta Forma Normal (4FN)

Requisitos:

- Debe cumplir con FNBC
- No debe tener dependencias multivaluadas no triviales
- Una tabla no debe contener dos o más relaciones independientes muchos-a-muchos

Ejemplo de violación:



Ilustración 11- Ejemplo violación 4FN - Fuente: Autor de contenido

Si un empleado puede tener múltiples habilidades y múltiples idiomas independientemente.

Solución (4FN):

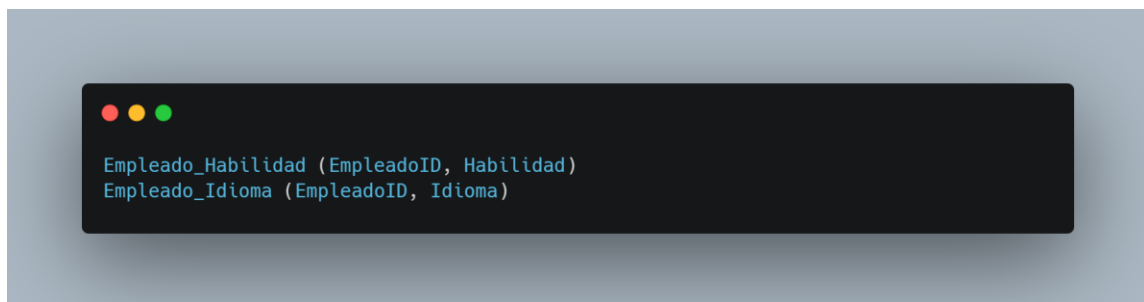


Ilustración 12- Ejemplo solución 4FN - Fuente: Autor de contenido

Quinta Forma Normal (5FN)

Requisitos:

- Debe cumplir con 4FN
- Elimina dependencias de unión (join dependencies)
- La tabla no puede descomponerse en tablas más pequeñas sin perder información

Es raramente necesaria en la práctica y se usa en casos muy específicos con relaciones complejas entre tres o más entidades.

3. Desnormalización

En ocasiones, deliberadamente se **desnormaliza** una base de datos (se viola la

normalización) para mejorar el rendimiento en sistemas de lectura intensiva. Esto implica:

- Duplicar datos estratégicamente
- Combinar tablas frecuentemente consultadas juntas
- Agregar columnas calculadas o redundantes
- Crear tablas de resumen

4. DDL y DML

SQL organiza sus comandos en sublenguajes especializados: DDL para definir estructuras, DML para manipular datos, DCL para controlar accesos, TCL para gestionar transacciones y DQL para consultas.

- **DDL (Estructura):** Define y altera objetos de la base de datos como tablas e índices con CREATE, ALTER y DROP.
- **DML (Datos):** Inserta, actualiza, elimina y selecciona datos mediante INSERT, UPDATE, DELETE y SELECT.
- **DCL (Data Control Language):** Gestiona permisos con GRANT (otorga accesos) y REVOKE (revoca privilegios), asegurando seguridad.
- **TCL (Transaction Control Language):** Controla transacciones con COMMIT (guarda cambios), ROLLBACK (deshace) y SAVEPOINT (puntos de guardado).
- **DQL (Data Query Language):** Enfocado en consultas con SELECT para recuperar datos sin modificarlos.

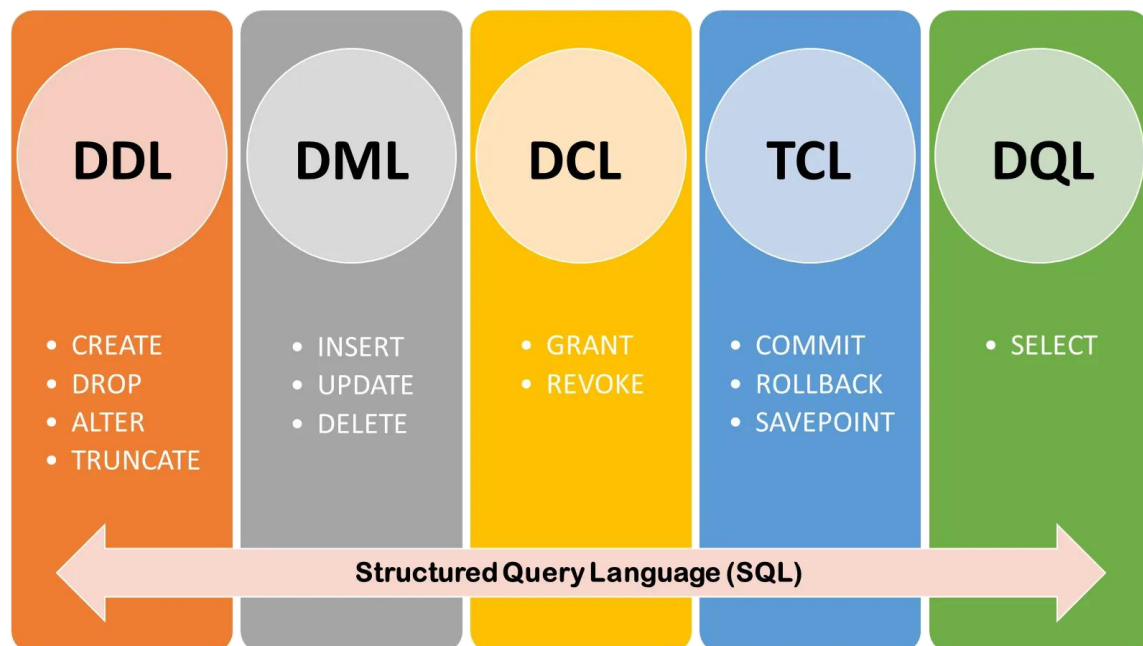


Ilustración 13- Sublenguaje SQL - Fuente: analyticsvidhya.com

3 Bases de datos

3.1 ¿Porque las bases de datos son tan importantes?

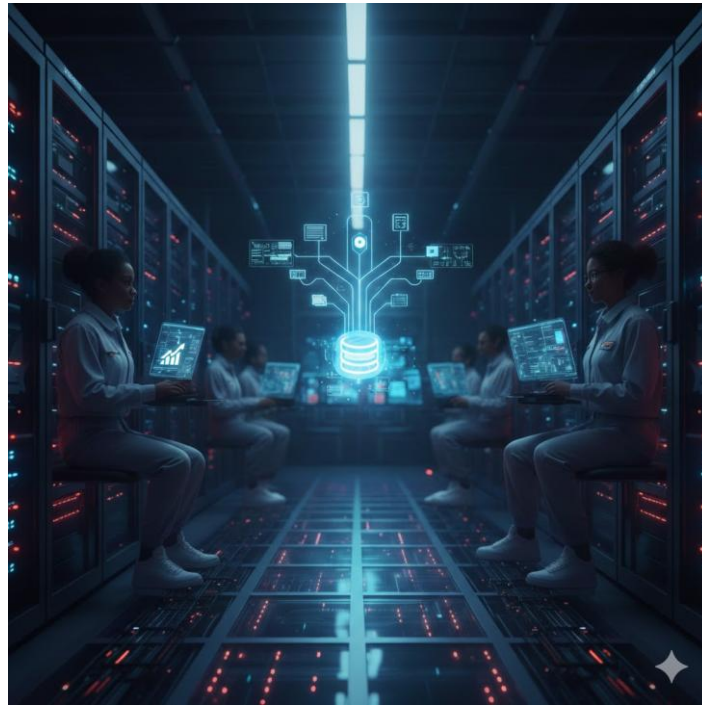


Ilustración 1 - Bases de datos - Fuente: Gemini Nano Banana Pro

Las bases de datos no son un “extra”; son el núcleo invisible que permite que la sociedad digital funcione sin colapsar. Si desaparecieran esta noche, el mundo entero notaría el apagón antes del amanecer.

1. **Son la única forma de convertir datos en tiempo real en decisiones en tiempo real:** Sin ellas un cajero no puede saber si tienes saldo; un hospital no puede saber que eres alérgico a un antibiótico; un satélite no puede re-orientar sus paneles solares. La base de datos es el sistema nervioso que lleva la “electricidad” de la información al órgano que debe actuar.
2. **Garantizan que la verdad sea única:** En cualquier organización hay cientos de copias de lo “mismo” (tu email, tu DNI, tu pedido). La base de datos impide que existan dos verdades contradictorias; sin ella, la empresa no sabría cuál es la real y cada departamento inventaría la suya.
3. **Hacen escalar lo humano:** Un empleado puede recordar 100 clientes; una base de datos, 100 millones. Amplifica la memoria humana sin línea de ruptura: el coste de acceder al dato número 1 es el mismo que acceder al dato número 1 000 000 000.
4. **Son el antídoto contra el caos de la concurrencia:** Miles de usuarios tocan la misma “fila” simultáneamente (entradas de un concierto, reservas de un vuelo).

El motor de base de datos secuencía esas operaciones para que no se vendan dos veces el mismo asiento ni se duplique el último euro de una cuenta.

5. **Permiten la inteligencia colectiva:** Recomendarte la siguiente canción, detectar un fraude bancario o predecir una tormenta solo es posible porque millones de eventos quedaron registrados con estructura y hora. La base de datos es la “cera” sobre la que se construyen los modelos de aprendizaje automático.
6. **Protegen el activo más valioso: la continuidad:** Copias, réplicas, respaldos en la nube: la base de datos garantiza que un incendio, un ransomware o un error humano no borre el pasado de una empresa ni la capacidad de seguir operando mañana.

4.0 Tipos de Bases de Datos

Tipo	Descripción	Bases de datos
Bases de Datos Relacionales (SQL)	Son las más tradicionales y utilizan tablas para organizar la información. Los datos se relacionan entre sí mediante claves primarias y foráneas. Son ideales cuando necesitas mantener la integridad de los datos y realizar consultas complejas con relaciones bien definidas.	• MySQL • PostgreSQL • Oracle • SQL Server
Bases de Datos NoSQL	Surgieron para manejar grandes volúmenes de datos no estructurados. Se dividen en varios subtipos: • Documentales • Clave-Valor • Columnares • Grafos	• MongoDB • CouchDB • Cassandra • HBase
Bases de Datos en Memoria	Almacenan todo en RAM para velocidad extrema. Se usan principalmente para cachés y procesamiento en tiempo real.	• Redis • Memcached

Bases de Datos de Series Temporales	Optimizadas para datos con marcas de tiempo. Esenciales en IoT, monitoreo de sistemas y análisis financiero.	• InfluxDB • TimescaleDB
-------------------------------------	--	--

Tabla 2 - Tipos de bases de datos - Fuente: Autor de contenidos

Introducción a MySQL

MySQL es un sistema de gestión de bases de datos relacional, open-source y gratuito, mantenido por Oracle. Usa tablas con filas y columnas, relacionadas mediante claves primarias y foráneas, y se controla con el lenguaje SQL. Destaca por su velocidad, fiabilidad y facilidad de uso, integrándose sin problemas con PHP, Python, Java y servidores web como Apache o Nginx. Soporta transacciones ACID, índices, vistas, triggers y procedimientos almacenados, y permite replicación maestro-esclavo y clustering para alta disponibilidad. Es ideal para aplicaciones web, desde blogs hasta tiendas online, y tiene versiones embebidas (MySQL Embedded) o en la nube (MySQL HeatWave). Su comunidad es enorme, con miles de tutoriales, foros y herramientas gráficas (phpMyAdmin, Workbench).

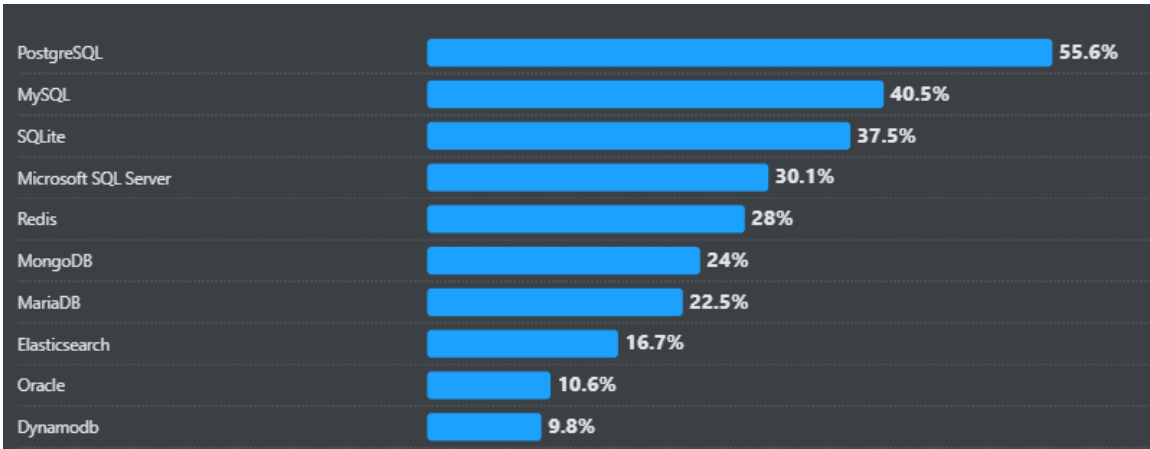


Ilustración 2 - Datos estadísticos - Fuente: Stackoverflow survey 2025

A pesar del crecimiento explosivo de PostgreSQL en los últimos años, los datos de la encuesta de Stack Overflow 2025 (donde MySQL se mantiene sólido en el segundo lugar con un 40.5%) demuestran que sigue siendo una pieza fundamental e inamovible del ecosistema tecnológico.

Características de MySQL

MySQL brilla especialmente en aplicaciones web, sistemas de gestión de contenidos (WordPress, Drupal), e-commerce, aplicaciones empresariales y cualquier proyecto que requiera transacciones ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad).

La base de datos MySQL ofrece todo un ecosistema de tipos de datos que se ajustan a las necesidades modernas, los tipos de datos los cuales son:

- **Tipos Numéricos:** Desde TINYINT hasta DOUBLE, incluyendo DECIMAL para valores monetarios donde la precisión es crítica.

Tipos Numéricos				
Tipo de Dato	Descripción	Rango	Tamaño	Uso Común
TINYINT	Entero muy pequeño	-128 a 127 (signed) / 0 a 255 (unsigned)	1 byte	Banderas, estados, edad
SMALLINT	Entero pequeño	-32,768 a 32,767 (signed) / 0 a 65,535 (unsigned)	2 bytes	Contadores pequeños
MEDIUMINT	Entero mediano	-8,388,608 a 8,388,607 (signed) / 0 a 16,777,215 (unsigned)	3 bytes	IDs de rango medio
INT	Entero estándar	-2,147,483,648 a 2,147,483,647 (signed) / 0 a 4,294,967,295 (unsigned)	4 bytes	IDs, cantidades generales
BIGINT	Entero grande	-9,223,372,036,854,775,808 a 9,223,372,036,854,775,807	8 bytes	IDs grandes, timestamps
DECIMAL(M,D)	Número decimal de precisión fija	Depende de M y D	Variable	Dinero, precios, cálculos precisos
FLOAT	Número de punto flotante	-3.402823466E+38 a 3.402823466E+38	4 bytes	Mediciones científicas
DOUBLE	Número de punto flotante doble precisión	-1.7976931348623157E+308 a 1.7976931348623157E+308	8 bytes	Cálculos científicos complejos

Ilustración 3 - Tipos Numéricos - Fuente: Autor de contenidos

- Tipos de Texto:** Desde CHAR para datos de longitud fija hasta LONGTEXT para contenido extenso. También incluye ENUM y SET para valores predefinidos.

Tipos de Cadena de Texto			
Tipo de Dato	Descripción	Longitud Máxima	Uso Común
CHAR(n)	Cadena de longitud fija	255 caracteres	Códigos fijos, abreviaciones
VARCHAR(n)	Cadena de longitud variable	65,535 caracteres	Nombres, emails, direcciones
TINYTEXT	Texto pequeño	255 caracteres	Notas cortas
TEXT	Texto mediano	65,535 caracteres (64 KB)	Descripciones, comentarios
MEDIUMTEXT	Texto grande	16,777,215 caracteres (16 MB)	Artículos, documentos
LONGTEXT	Texto muy grande	4,294,967,295 caracteres (4 GB)	Libros, contenido extenso
ENUM	Lista de valores predefinidos	Hasta 65,535 valores distintos	Estados, categorías fijas
SET	Conjunto de valores predefinidos	Hasta 64 valores distintos	Tags, múltiples opciones

Ilustración 3 - Tipos de Cadena de Texto - Fuente: Autor de contenidos

- **Tipos de Fecha/Hora:** DATE, TIME, DATETIME, TIMESTAMP y YEAR, cada uno con sus casos de uso específicos.

Tipos de Fecha y Hora				
Tipo de Dato	Descripción	Formato	Rango	Uso Común
DATE	Solo fecha	YYYY-MM-DD	1000-01-01 a 9999-12-31	Fechas de nacimiento, eventos
TIME	Solo hora	HH:MM:SS	-838:59:59 a 838:59:59	Horarios, duraciones
DATETIME	Fecha y hora	YYYY-MM-DD HH:MM:SS	1000-01-01 00:00:00 a 9999-12-31 23:59:59	Registros de actividad
TIMESTAMP	Marca de tiempo	YYYY-MM-DD HH:MM:SS	1970-01-01 00:00:01 UTC a 2038-01-19 03:14:07 UTC	Creación/modificación de registros
YEAR	Solo año	YYYY	1901 a 2155	Años de fabricación, períodos

Ilustración 4 - Tipos de Fecha y Hora - Fuente: Autor de contenidos

- **Tipo JSON:** Para almacenar datos en formato JSON nativamente.

Tipo JSON

Tipo de Dato	Descripción	Uso Común
JSON	Datos en formato JSON	Configuraciones, datos semi-estructurados, APIs

Ilustración 5- Tipo JSON - Fuente: Autor de contenidos

MySQL ofrece, además de los tipos clásicos, familias especializadas: datos binarios (BLOB, LONGBLOB) para archivos y multimedia, y espaciales (GEOMETRY, POINT, POLYGON) que cumplen con el estándar OGC para información geográfica.

Estos tipos amplían el alcance del motor, pero su manejo exige precaución: pueden generar errores de conversión, índices corruptos o cuellos de botella de almacenamiento si no se dimensionan correctamente.

Operaciones CRUD y su sintaxis en MySQL

CRUD son las 4 operaciones básicas en bases de datos:

1. CREATE (Crear) - INSERT

Sirve para agregar nuevos registros a una tabla.

```
INSERT INTO tabla (columna1, columna2, columna3)
VALUES (valor1, valor2, valor3);
```

Ilustración 6 - CREATE (Crear) - INSERT - Fuente: Autor de contenidos

2. READ (Leer) - SELECT; Su majestad el “SELEC”

Sirve para consultar y ver datos de la tabla.

```
SELECT columnas FROM tabla WHERE
```

Ilustración 7 - Su majestad el “SELEC” - Fuente: Autor de contenidos

3. UPDATE (Actualizar) - UPDATE

Sirve para modificar registros existentes.

```
UPDATE tabla
SET columna1 = nuevo_valor
WHERE condicion;
```

Ilustración 8 - UPDATE (Actualizar) - UPDATE - Fuente: Autor de contenidos

4. DELETE (Eliminar) - DELETE

Sirve para borrar registros de la tabla.

DELETE FROM tabla **WHERE** condición;

Ilustración 8 - DELETE (Eliminar) - DELETE - Fuente: Autor de contenidos

1.1. Funciona de agregación y agrupación básicas de MySQL

Las funciones de agregación en MySQL realizan cálculos sobre conjuntos de filas para devolver un valor único, mientras que el agrupamiento (GROUP BY) organiza los datos en grupos basados en columnas específicas.

Función	Descripción	Sintaxis	Ejemplo
COUNT(*)	Cuenta todas las filas del grupo, incluyendo duplicados y NULL.	COUNT(*)	SELECT COUNT(*) FROM usuarios;
COUNT(columna)	Cuenta filas no-NULL en la columna especificada.	COUNT(nombre)	SELECT COUNT(nombre) FROM usuarios;
COUNT(DISTINCT columna)	Cuenta valores únicos no-NULL.	COUNT(DISTINCT ciudad)	SELECT COUNT(DISTINCT ciudad) FROM usuarios;
SUM(columna)	Suma valores numéricos.	SUM(salario)	SELECT SUM(salario) FROM empleados;
AVG(columna)	Calcula el promedio de valores numéricos.	AVG(precio)	SELECT AVG(precio) FROM productos;
MIN(columna)	Encuentra el valor mínimo.	MIN(fecha)	SELECT MIN(fecha_nacimiento) FROM usuarios;
MAX(columna)	Encuentra el valor máximo.	MAX(edad)	SELECT MAX(edad) FROM usuarios;
GROUP_CONCAT(columna)	Concatena valores en una cadena separada por comas.	GROUP_CONCAT(nombre SEPARATOR ', ')	SELECT GROUP_CONCAT(nombre) FROM productos;

Tabla 2 - Funciones de Agregación - Fuente: Autor de contenidos

Cláusula GROUP BY

Agrupar filas con valores iguales en las columnas especificadas, aplicando funciones de agregación por grupo.

- Sintaxis básica: *SELECT columna, SUM(otra_columna) FROM tabla GROUP BY columna;*
- Ejemplo: *SELECT departamento, AVG(salario) FROM empleados GROUP BY departamento;* calcula el salario promedio por departamento.
- Se combina con HAVING para filtrar grupos (no filas individuales, como WHERE).

Recursos de Apoyo



Video

[Creación y gestión de bases de datos ejercicio práctico.](#)



Material de Lectura

[Guía de normalización de base de datos.](#)



Material de Lectura

[Creación de diagramas ER.](#)